

Data Structure (week 6)

講師：陳俊豪

我們已經學會了兩種強大武器，然後呢？

	Static Array	Dynamic Array	Singly Linked List	Doubly Linked List
記憶體配置	連續	連續 (會搬家)	散落各處 (會牽手)	散落各處 (多長一隻手)
空間	固定	彈性 (會搬家)	彈性	彈性 (增加成本)
存取資料	$O(1)$	$O(1)$	$O(N)$	$O(N)$
Insert 問題	$O(N)$	$O(N)$	$O(1)$	$O(1)$
往回找資料	$O(1)$	$O(1)$	$O(N)$	$O(1)$

我們已經學會了兩種強大武器，然後呢？

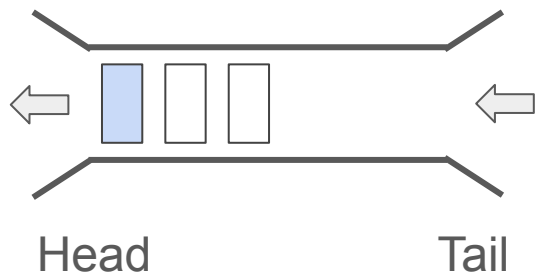
- 現實中大多通電的東西都和程式有關，也都有資料結構身影
- 但在真實世界中，Array 與 Linked List 對應兩種直覺的系統：「隨機抽查」(Random Access) 和「隨機插隊」(Random Insertion)
 - Random Access：劃位系統、資料庫
 - Random Insertion：上/下一個系統
- 但這樣足夠嗎？

Queue

佇列

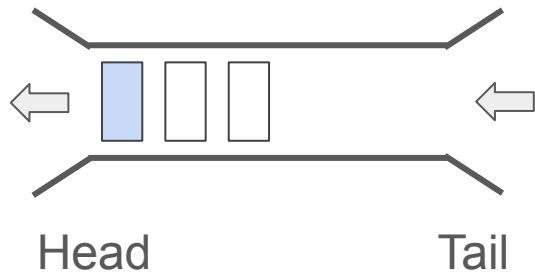
Queue (佇列)

- 主要對應現實中的「**排隊系統**」
 - 世界許多系統為了 **維持秩序**，是禁止插隊的！
 - 核心精神：First-In-First-Out (FIFO)
 - p.s. 與會計有 "實體" 上的區別
- 專屬功能 (Queue 的世界只有兩個合法動作)：
 - **Enqueue** (入列)
 - 資料只能從「隊伍尾端 (Tail)」加入
 - **Dequeue** (出列)
 - 資料只能從「隊伍前端 (Head)」離開



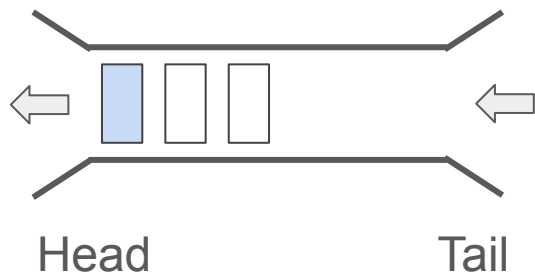
Queue (佇列)

- 主要對應現實中的「**排隊系統**」
 - 世界許多系統為了 **維持秩序**，是禁止插隊的！
 - 核心精神：First-In-First-Out (FIFO)
 - p.s. 與會計有 "實體" 上的區別
- 專屬功能 (Queue 的世界只有兩個合法動作)：
 - Enqueue (入列)
 - Array vs Linked List
 - Dequeue (出列)
 - Array vs Linked List



Queue (佇列)

- 主要對應現實中的「**排隊系統**」
 - 世界許多系統為了 **維持秩序**，是禁止插隊的！
 - 核心精神：First-In-First-Out (FIFO)
 - p.s. 與會計有 "實體" 上的區別
- 專屬功能 (Queue 的世界只有兩個合法動作)：
 - Enqueue (入列)
 - Array $O(1)$ 、 $O(N)$ vs Linked List $O(1)$
 - Dequeue (出列)
 - Array $O(N)$ vs Linked List $O(1)$



Queue (佇列)

- 主要對應現實中的「**排隊系統**」
 - 世界許多系統為了 **維持秩序**，是禁止插隊的！
 - 核心精神：First-In-First-Out (FIFO)
 - p.s. 與會計有 "實體" 上的區別
- 專屬功能 (Queue 的世界只有兩個合法動作)：
 - Enqueue (入列)
 - **$O(1)$**
 - Dequeue (出列)
 - **$O(1)$**
 - Linked List 太強大, 而 Array 又太限縮, **Queue 是折衷上有「秩序」又有「效率」**
 - 再聯想一下 Python deque 套件

Queue (佇列) : Overflow & Underflow

- 儘管設計出看似安全又效率的 Queue 仍有它自己的問題
- Queue **Overflow**
 - 演唱會搶票系統
 - 一秒鐘湧入 10 萬個購票請求 (?), 但伺服器一秒只能處理 1000 個請求 (?)
 - BlockingQueue、**Dynamic Resizing**
- Queue **Underflow**
 - YouTube / Netflix 的「轉圈圈」
 - 影片入載的速度 (?), 沒有看影片的速度快 (?)

補充：比賽資訊 ([獎金獵人](#))



- 報名：每年的 9 月底
- 報名隊伍：約 100 組
- 進複賽：固定 20 組
- 得獎：前 7 + 13 佳作

Queue : Multiple Queues (多重佇列)

- 只有一個 Queue 總是會不夠 > Multiple Queues
 - 超市結帳模式
- 那回到 Queue Overflow (演唱會搶票系統) 要怎麼解決？
 - 請嘗試設計架構

Queue : Deque (雙向佇列)

- 打破了標準 Queue「只能 Tail 進、Head 出」的限制
- 它允許你從「頭部」與「尾部」同時進行 $O(1)$:
 - 新增 (append, appendleft)
 - 刪除 (pop, popleft)
- Deque 是 Python 處理頭尾資料的神器, 能同時完美勝任 Queue 與 Stack
 - Python 的 collections.deque 能完全實踐這些資料結構
 - Python 沒有內建叫做 Linked List
 - 上次說到 deque 沒有 "往回找資料" 的能力 why ?
 - 而 queue 根本沒有 insert 反倒 deque 有 why ?

Priority Queue (優先佇列)

- 目標：讓特殊單位 (VIP) 可以瞬間「插隊」到第一位
 - 目前只有 Enqueue 和 Dequeue 還無法實現，需要未來教 Heap 結構時才能實現
- Linked List 無法實作的原因
 - 不知道 VIP 單位在哪 ex. [10, 10, 10, VIP, 10, 10]
 - Search 需要 $O(N)$
 - 但我們希望的是 VIP 要能瞬間到第一位，不用查票



In-class exercises 1

- 情境：

遊戲剛開局，玩家建立了一座初階兵營。玩家連續點擊了三個生產指令，接著系統經過了一段時間，完成了第一個單位的生產。請你用 Python 模擬這個過程，完成以下 TODO。(見 Colab)



In-class exercises 2

- 情境：

初階兵營很容易壞掉！如果玩家亂點把記憶體塞爆 (Overflow), 或是兵營明明沒事做系統卻硬要拿兵出來 (Underflow), 遊戲就會當機閃退。請你加上 Queue 「最多只能排 2 個人」的限制, 以及「空兵營不能生產」的防呆機制。

HW3-1：智能派單 (Smart Enqueue) (70%)

- 請使用 for loop 讀取 orders 裡的每一個訂單，並嚴格執行以下生產邏輯：
 - **資源檢核**：檢查 player_gold 是否足夠支付 cost。不夠請 print「黃金不足，無法生產 **XX**」。
 - **負載平衡**：若買得起，將訂單放進排隊人數較少的 Queue (一樣長則優先放 A)。
 - **Overflow 防禦**：A 和 B 的產線上限都是 2 個。如果兩個兵營都滿了，即使玩家有錢也不能生產，請 print「產線全滿！**XX** 訂單拒絕」。
 - **Underflow 防呆**：出列前必須檢查產線是否有單位，若是空的請 print「**X** 廠沒東西可做」。
 - **Dequeue**：每當偶數回合，系統自動觸發 Dequeue 出列
 - **允許空單 {} 存在**：每當回合數是 2 的倍數，系統自動觸發雙廠 Dequeue，請 print「玩家本回合無動作，單純推進時間」。
 - **系統邏輯順序**：需自行判斷

HW3：結果展示

- [測資1] 透過以下不同回合的輸出結果，觀察並完成答案。

--- 第 0 回合 ---

A 廠沒東西可做 (Underflow 防護成功)

B 廠沒東西可做 (Underflow 防護成功)

劍士 分派至 A 廠 (剩餘黃金: 130)

A: ['劍士'] | B: []

--- 第 2 回合 ---

A 廠生產完成：劍士 出列！

B 廠生產完成：弓手 出列！

騎士 分派至 A 廠 (剩餘黃金: 50)

A: ['騎士'] | B: []

HW3-2 (初級) Two Sum (30%)

- 給定 `nums = [2,7,11,15]; target = 9;`, 請實現一個函數 input 為 nums 和 target, output 為在 nums 中任兩個相加等於 target 的 index, ex. $2 + 7 = 9$, output 就會是 [0, 1] (output 輸出要求是 array, 順序都可以) ?
 - 接受暴力解
 - 效率解 (再 +30%)

HW4 - Bonus: Take-Home Challenge

- 讓 AI 用 HTML/CSS/JS 視覺化出 HW3-1 的結果
- 素材可以網路上下載 or 使用 nano banana 2 生成 (gemini)
- 要有「開始」與「重製」按鈕
- 加分範圍 0~1

補充資訊

